

Отчет по выполнению итогового задания модуля 4 по дисциплине ETL-процессы

Работу выполнила Евсеева Полина

В ходе выполнения работы я последовательно реализовала несколько этапов обработки и визуализации данных в сервисах Yandex Cloud. В работе использовались YDB, Object Storage, Managed Service for Apache Airflow, Managed Service for Apache Kafka, Yandex Data Processing, Yandex Query и DataLens.

Задание 1

На первом этапе я создала базу данных в YDB и бакет в Object Storage. Затем в базе данных была создана таблица `transactions_v2` с помощью скрипта `create table.yql`. После этого я сгенерировала данные и загрузила их в таблицу через командную строку Yandex CLI. Далее были созданы эндпоинты для трансфера, после чего я создала сам трансфер и активировала его (скриншот выполнения приложен в репозитории). В результате данные успешно перенеслись из таблицы в бакет в формате JSON, а итоговый файл был сохранён с именем `part-1781279665-c21f969b.00000.json`.

Задание 2

Во втором задании я сначала написала скрипт `generate_applications.py`, запустила его и получила CSV-файл с исходными данными для последующей обработки. Затем был создан скрипт `job.py`, который читает CSV-файл из бакета, выполняет простую очистку данных и сохраняет результат в формате Parquet в выходной бакет. Для выполнения этого задания я также создала отдельные бакеты: `airflow-bucket` для Managed Service for Apache Airflow, `2-task` для исходных данных и `2-task-output` для результатов обработки.

После подготовки хранилища я создала облачную сеть и настроила NAT-шлюз. Затем был развернут кластер в Managed Service for Apache Airflow, после чего я вошла в его веб-интерфейс и выполнила дополнительную настройку. Также я создала кластер Hive Metastore, который потребовался для корректной работы пайплайна. Далее я написала DAG `dag_dataproc_applications.py` и разместила его в папке `dags` в бакете `airflow`, откуда он был автоматически подхвачен системой Airflow. Я запустила DAG и убедилась, что он выполнен успешно. Результат работы был проверен по содержимому бакета с выходными данными, где появился обработанный Parquet-файл. В репозитории на гитхабе приложены скриншоты из Airflow и результат отработки дага в бакете

Задание 3

Для третьего задания сначала я создала сеть, подсеть, NAT-шлюз, группу безопасности и бакет. После этого был развернут кластер Yandex Data Processing, а также кластер

Managed Service for Apache Kafka. В рамках Kafka я создала топик и отдельного пользователя для подключения к кластеру.

Далее я написала PySpark-скрипт `generate_kafka_json.py` для генерации JSON-данных. После запуска этого задания я получила статус `Done`, что подтвердило успешное создание тестовых данных. Затем я подготовила второй PySpark-скрипт `kafka-job.py`, создала второе задание и запустила его в обработку. Оно также завершилось успешно со статусом `Done`, а результат выполнения был сохранён в бакете в виде Parquet-файла (скриншот обработки приложен к результатам в репозитории).

Задание 4

В четвертом задании я зашла в DataLens и создала подключение типа YDB для данных из первого задания. На основе этого подключения был создан датасет, после чего на вкладке «Поля» я настроила типы данных, чтобы они корректно использовались в визуализациях и вычисляемых показателях.

После подготовки датасета я создала дашборд и добавила в него несколько графиков. Для первого дашборда были использованы следующие визуализации: динамика количества звонков по дням, топ-5 регионов по количеству звонков, распределение звонков по статусам, средняя длительность звонка по типам кампании и диаграмма общей длительности звонков. Эти графики позволяют оценить динамику активности, выявить наиболее загруженные регионы, сравнить статусы звонков и проанализировать длительность разговоров в разрезе кампаний.

Затем я перешла к дашборду для второго задания и создала новый датасет через подключение к бакету с результатами обработки с помощью Yandex Query. В этот дашборд были добавлены графики по проценту одобренных и отклонённых заявок, динамике одобрений и отказов по дням, одобрению по уровню риска, времени обработки по каналу и вероятности одобрения в зависимости от `credit_score`. Такой набор визуализаций помогает оценить эффективность кредитного конвейера и определить факторы, влияющие на решение по заявке.

После этого я настроила дашборд для третьего задания, также используя подключение через Yandex Query. Так как данные были схожи с предыдущим заданием, чарты тут тоже перекликаются. Также хочу отметить, что данные для заданий были сгенерированы и выдавали довольно схожие результаты, что можно увидеть на графиках, в реальных данных динамика бы наблюдалась сильнее, но на выполнение задания это не влияет. В третий дашборд были добавлены графики по количеству заявок по регионам, доле одобрения по уровню риска, объёму займов по сроку кредита и таблица с характеристиками по уровням риска.

Структура репозитория

Общая структура

```
etl-final-module-4/  
├─ 1_task/  
├─ 2_task/  
├─ 3_task/  
├─ 4_task/  
└─ report/
```

Содержимое папок

Внутри каждой папки размещены файлы, относящиеся именно к конкретному заданию. При необходимости используются подпапки:

```
1_task/  
├─ scripts/  
├─ results/  
└─ data/
```

```
2_task/  
├─ scripts/  
├─ dags/  
├─ results/  
└─ data/
```

```
3_task/  
├─ scripts/  
└─ results/
```

```
4_task/  
└─ results/
```

```
report/  
└─ Отчет_по_итоговому_заданию_4_модуля_Евсеева_Полина.pdf
```